



**National
Supercomputing
Mission**



**Centre for HPC
Upskilling &
Knowledge-sharing**



**Centre for
Development of
Advanced Computing**

OPENMP PROGRAMMING

Ankit Mishra
ankitkumarbm@cdac.in
C-DAC, Bengaluru



CONTENT

Introduction to OpenMP

OpenMP Programming Model

OpenMP Programs

OpenMP Use Cases

Conclusion

References



INTRODUCTION TO OPENMP

- OpenMP is an Application Program Interface (API).
- The API supports C, C++ and FORTRAN.
- OpenMP is designed for multi-processor/shared memory machines.



INTRODUCTION TO OPENMP : PROCESS AND THREAD CONCEPTS

Process:

“The Program in execution is called Process.”

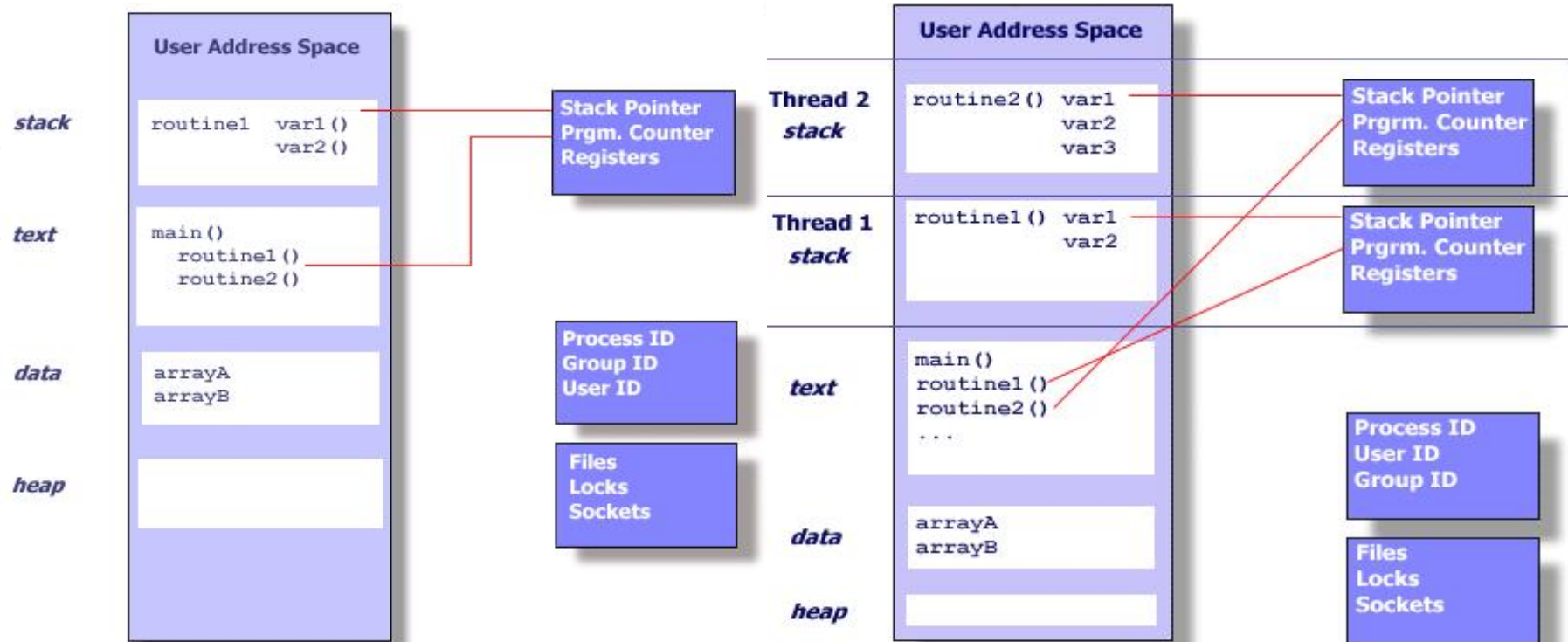
Thread:

“The smallest unit of execution within a process is called Thread.”

- Each thread has its own stack, register set, program counter.

INTRODUCTION TO OPENMP : PROCESS AND THREAD CONCEPTS

September 22 to 25, 2026, OpenMP Programming, C-HUK-ACC





INTRODUCTION TO OPENMP : PROCESS AND THREAD CONCEPTS

Multi-threading:

“The execution of multiple threads concurrently within a single process is called Multi-threading.”

- The Threads created are called team of threads.
- The creation of multiple thread is called thread spawning.



INTRODUCTION TO OPENMP : PROCESS AND THREAD CONCEPTS

Master Thread:

“Each process contains at least one thread is called Master Thread.”

Worker Thread:

“The thread spawned out of Master thread in multi-threading is called Worker thread.”



INTRODUCTION TO OPENMP : PROCESS AND THREAD CONCEPTS

- **NOTE:**

Master Thread is primary thread in a multithreaded program responsible for spawning and coordinating Worker Threads.

- OpenMP provides Shared Memory based thread-based parallelism.

INTRODUCTION TO OPENMP :

OPENMP STACK

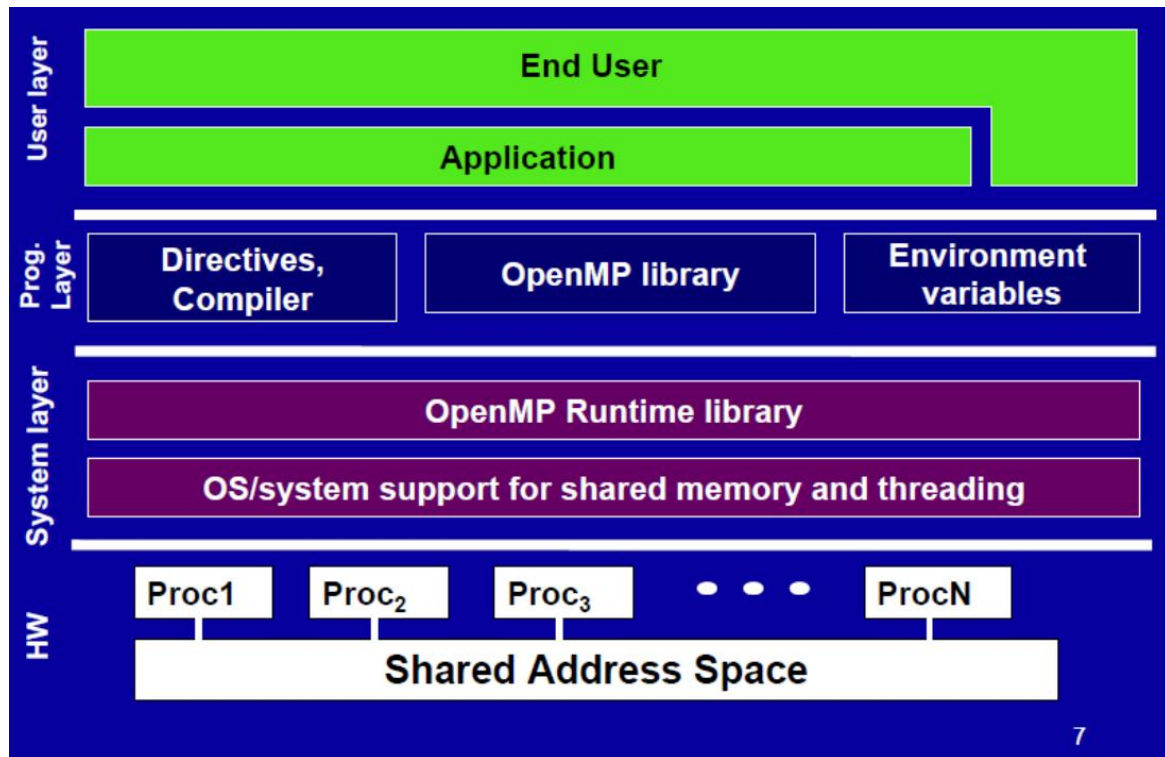


Image: OpenMP Stack^[4]

OPENMP PROGRAMMING MODEL : FORK – JOIN PARALLELISM

- OpenMP uses Fork – Join Model for Parallelization.

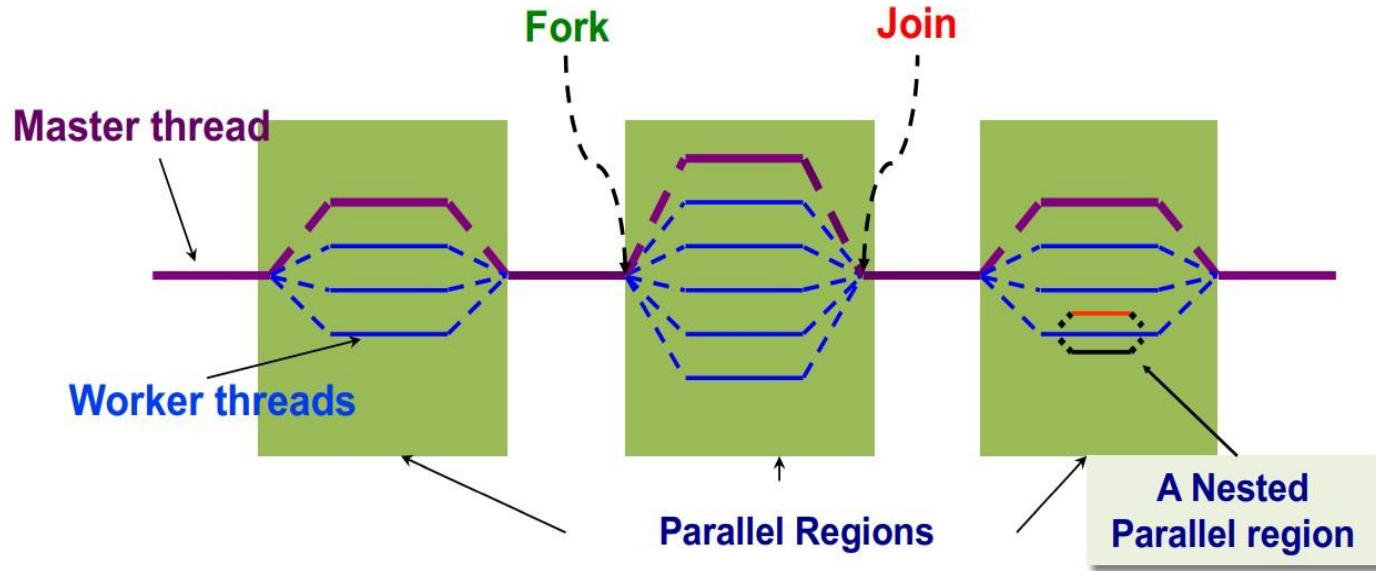


Image: OpenMP Fork – Join Programming Model

OPENMP PROGRAMMING MODEL : OPENMP MEMORY MODEL

- Threads can communicate by reading/writing variables in the common address space.
- Data is private or shared.
- Private accessed only by owned threads.
- Shared data accessible by all threads.

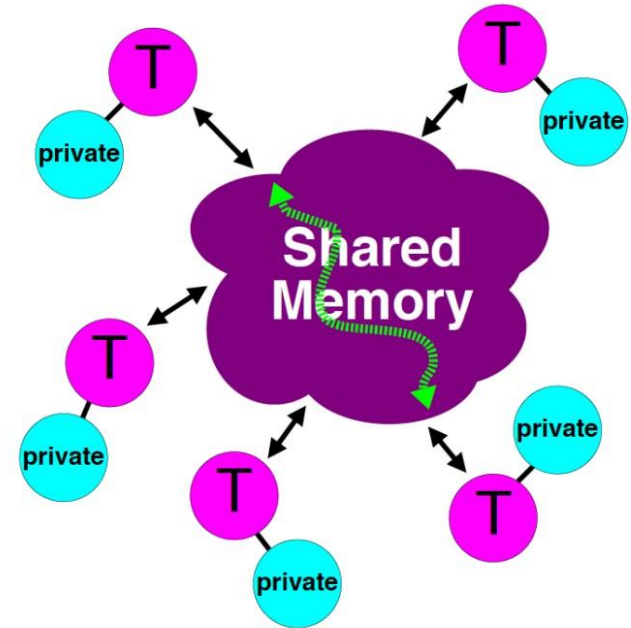


Image: Memory Model



OPENMP PROGRAMMING MODEL : SERIAL PROGRAM



```
#include <iostream>
```

```
int main()
```

```
{
```

```
int ID ;
```

```
    std::cout << "Hello my ID is : " << ID << std::endl;
```

```
return(0);
```

```
}
```



OPENMP PROGRAMMING MODEL : OPENMP PROGRAM SKELETON

FORMAT:

Sentinel	directive-name	[clause ...]	newline
Required for all OpenMP C/C++ Directives #pragma omp	A valid OpenMP directive. Must appear after the pragma and before any clauses.	Optional. Clauses can be in any order, and repeated as necessary unless otherwise restricted.	Required. Proceeds the structured block which is enclosed by this directive.



OPENMP PROGRAMMING MODEL : OPENMP PROGRAM SKELETON

C/C++ Compiler Directive Format

sentinel **directive-name** clause

Example

#pragma omp parallel for



OPENMP PROGRAMMING MODEL : OPENMP PROGRAM SKELETON

```
#include <iostream>
```

```
#include <omp.h>
```

```
int main() {
```

```
    omp_set_num_threads (4);
```

```
    //continue to next slide...
```



OPENMP PROGRAMMING MODEL : OPENMP PROGRAM SKELETON

```
#pragma omp parallel
```

```
{
```

```
    int ID = omp_get_thread_num();
```

```
    std::cout << "Hello my ID is : " << ID << std::endl;
```

```
}
```

```
    return (0);
```

```
}
```



COMPILE AND EXECUTION OF OPENMP PROGRAM

GNU compiler command to compile openmp program

```
cmd> g++ -fopenmp demo.c -o demo.out
```

Linux command to execute openmp program

```
cmd> ./demo.out
```

'P A R A L L E L' CONSTRUCT

- The number of threads in a parallel region is determined by the following factors, precedence in ascending order:
 - 1) Implementation default - usually the number of CPUs on a node, though it could be dynamic.
 - 2) Setting of the `OMP_NUM_THREADS` environment variable





'P A R A L L E L' CONSTRUCT

- 3) Use of the `omp_set_num_threads()` library function
- 4) Setting of the `NUM_THREADS` clause
- 5) Evaluation of the `IF` clause

NOTE:

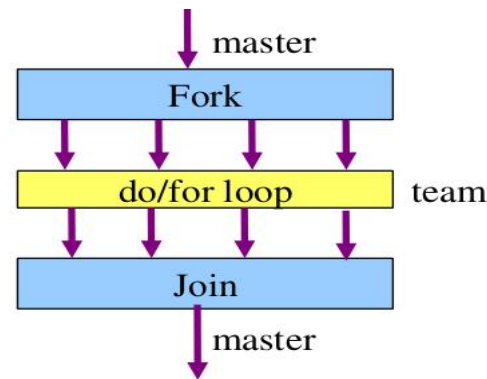
Threads are numbered from 0 (master thread) to N-1

WORK SHARING CONSTRUCT : 'for'

❖ OpenMP is usually used to parallelize loops:

- Find your most time consuming loops.
- Split them up between threads.

Split-up this loop between multiple threads



```
void main()
{
    double Res[1000];

    for(int i=0;i<1000;i++) {
        do_huge_comp(Res[i]);
    }
}
```

Sequential Program

#include "omp.h"

void main() Image: 'fork' construct

```
{
    double Res[1000];
    #pragma omp parallel for
    for(int i=0;i<1000;i++) {
        do_huge_comp(Res[i]);
    }
}
```

Parallel Program



WORK SHARING CONSTRUCT : 'for'

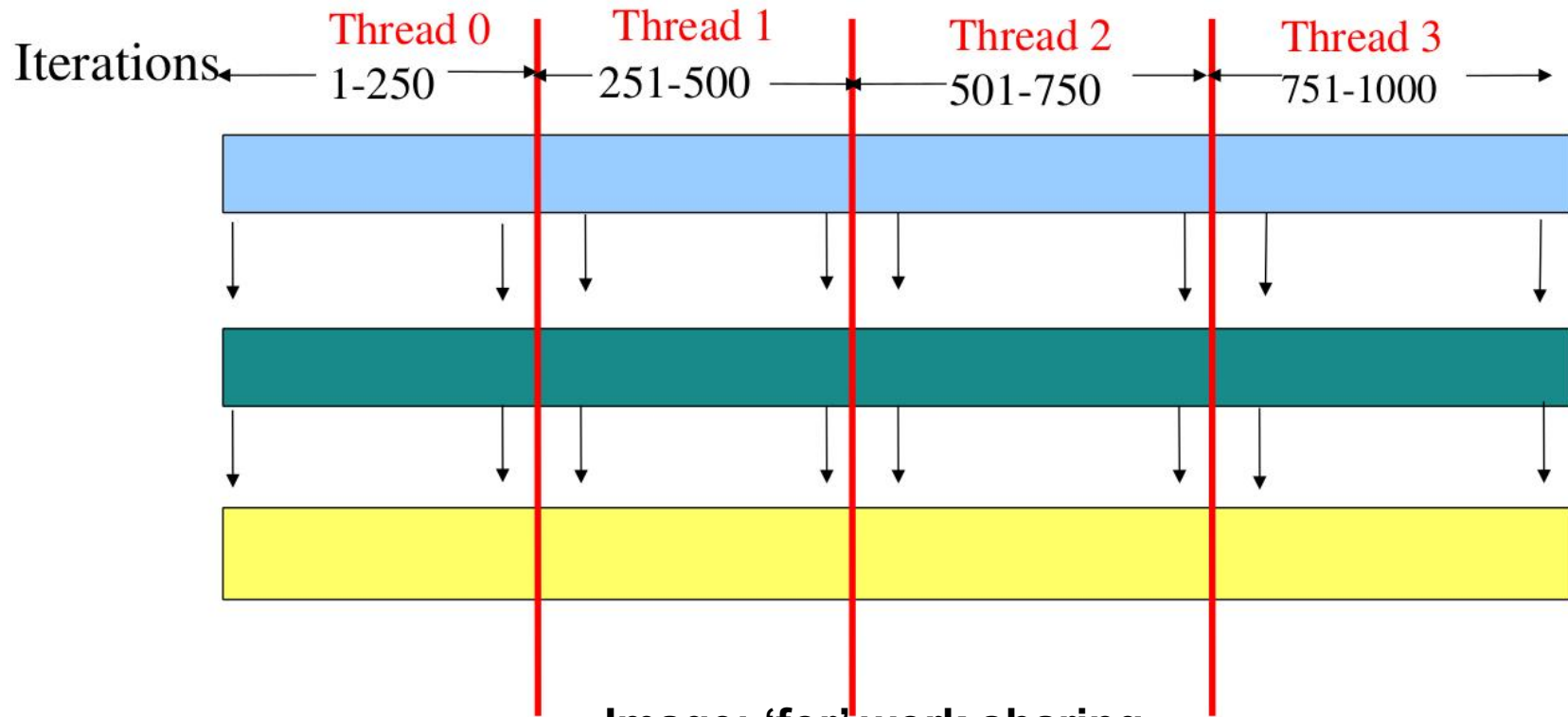


Image: 'for' work sharing

WORK SHARING CONSTRUCT 'for'

for directive identifies the iterative work-sharing construct.

**#pragma omp for [clause[[,]clause]...] new-line
for-loop**

WORK SHARING CONSTRUCT : 'for'

for (index = start ; index < end ; increment_expr)

- 1) it must be possible to determine the number of loop iterations before execution
- 2) no while loops



WORK SHARING CONSTRUCT : 'for'

- 3) increment must be the same each iteration
- 4) all loop iterations must be done
- 5) It is illegal to branch (goto) out of a loop associated with a for directive

USE CASE: MATRIX MULTIPLICATION

Input: Given two matrices A ($m \times n$) and B ($n \times p$) to be multiplied.

Output: Compute the matrix product C ($m \times p$) where $C = A * B$





USE CASE: MATRIX MULTIPLICATION

Computation:

each loop preceded by OpenMP construct

Outer loop (i): Iterate over the rows of matrix A (m rows).

Middle loop (j): Iterate over the columns of matrix B (p columns).

Inner loop (k): Iterate over the common dimension (n) shared by matrices A and B.

Set $C[i][j] = \text{sum of } (A[i][k] * B[k][j])$ for $k = 0$ to $n-1$



CONCLUSION

- Parallelization using OpenMP is explained.
- Various OpenMP Constructs are discussed with examples.



REFERENCES



- [1]. Parallel Programming in C using OpenMp and MPI- Micheal J Quinn
- [2]. HyPack workshop, C-DAC - http://www.cdac.in/index.aspx?id=ev_hpc_hypack13_m1_mcore_openmp
- [3]. Lawrence Livermore National laboratory – Training Material
https://computing.llnl.gov/?set=training&page=index#training_materials
- [4]. OpenMP by Tim Mattson, http://openmp.org/mp-documents/Intro_To_OpenMP_Mattson.pdf
- [5]. <http://openmp.org/mp-documents/omp-hands-on-SC08.pdf>
- [6]. OpenMp by Example - http://people.math.umass.edu/~johnston/PHI_WG_2014/OpenMPSlides_tamu_sc.pdf
- [7]. OpenMP Application Program Interface -<http://www.openmp.org/mp-documents/OpenMP3.1.pdf>
- [8]. OpenMP Application Program Interface - <http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf>
- [9]. <https://www.nersc.gov/assets/Uploads/XE62011OpenMP.pdf>
- [10]. www.clear.rice.edu/comp422/lecture-notes/comp422-2015-Lecture7-OpenMP.pdf



THANK YOU ANY QUESTION?